CHAPTER 7

Serving Requests with Phoenix

Phoenix is the biggest name in the Elixir ecosystem. If you've heard of Elixir before this book, there's a high chance that you've heard about Phoenix or Phoenix LiveView. We can't cover everything that Phoenix can do in a single chapter, but you'll learn enough to be dangerous with it.

Phoenix is a web-development framework that uses the Model-View-Controller (MVC) design pattern. It's well-integrated into the Elixir ecosystem primarily due to it being the most widely used framework in Elixir. On the surface, it's similar to Ruby on Rails, but it's fundamentally different in its goals and philosophy.

We'll go over the basics of Phoenix in this chapter. You'll serve web requests with controllers, use an asset pipeline for static assets, and learn about components for a clean UI. We'll use the Ecto-based data layer that we wrote in the previous chapter in order to read and write data.

Finally, we'll finish the chapter by looking at what else Phoenix can do. Phoenix's primary use case is web requests, but there are other offerings such as Phoenix Channels, Phoenix LiveView, and the authentication generator. We won't go deep into these topics, but you can continue learning about them in the official docs.

Let's start by going over what Phoenix is and how it compares to Rails!

Explore the Foundations of Phoenix

Phoenix is Elixir's most popular web-development framework. On the surface, this seems like it would be a large library with a ton of code, but it's actually fairly lightweight. (As of writing this paragraph, there are 46 files in the main `lib/phoenix` folder of the project.) This simplicity means that it's relatively quick to learn how to effectively use Phoenix to write web applications.